DATA SCIENCE

Lesson Plan

Python & SQL for Data Science

Complete Coverage

WEEKLY OVERVIEW

Data Science Roadmap — Python & SQL for Data Science, covering every sub-topic from core Python through advanced SQL analytics. By the end of this week, you will be able to load, clean, manipulate, and query real-world data using industry-standard tools.

Topic Area	Key Deliverable
Core Python Fundamentals	Mini programs with loops, functions, OOP
NumPy & Data Structures	Array operations & matrix manipulation
Pandas — Data Manipulation	EDA on Titanic dataset
SQL — Beginner to Intermediate	20+ queries on e-commerce DB
SQL — Advanced + Python + SQL Integration	Full analytical SQL report
Python Libraries Review + Project	COVID-19 dataset analysis project

THE THINGS NEED TO DO BEFORE CLASS

Students who arrive prepared get 3–5× more value from every class. The pre-class work is NOT optional — it is the foundation on which each day's session is built.

ONE-TIME ENVIRONMENT SETUP

Everyone must have the following installed and working BEFORE the first class:

Tool	Why It Is Needed	Where to Get It
Anaconda Distribution	Installs Python 3.11+ and 100+ science libraries in one click	anaconda.com — download Individual Edition
Jupyter Notebook	Interactive coding environment — runs code cell by cell	Included in Anaconda; launch with: jupyter notebook
VS Code	Professional code editor for writing .py scripts	code.visualstudio.com — install Python extension
Git + GitHub account	Version control — backs up all work done in class	git-scm.com and github.com — free accounts
Kaggle account	Download datasets used in class projects	kaggle.com — free registration

📖 CONCEPTS TO REVIEW BEFORE CLASS

You should spend 45–60 minutes before each class reviewing the following:

Core Python Basics

- What is a variable? What data types exist in Python? (int, float, str, bool, list, dict)
- What is a loop? Understand the idea of repeating code for each item in a list
- What is a function? Understand input → process → output
- Watch: 'Python in 100 seconds' on YouTube (Fireship channel) — just to get the big picture
- Read: Python.org Tutorial Sections 1–3 (free online)

NumPy & Arrays

- Understand what a matrix is (rows and columns of numbers)
- Recall from Phase 2: what is a vector? What is a dot product?
- Watch: '3Blue1Brown — Essence of Linear Algebra, Chapter 1' (YouTube, free)
- Install confirmation: open Anaconda terminal and type: import numpy as np — should not give any error

Pandas & Data Manipulation

- Download the Titanic dataset from Kaggle (titanic/train.csv)
- Open it in Excel to understand the structure: what are the columns? What does each row mean?
- Think about: what questions would you ask about this data? Read: 'What is Exploratory Data Analysis?' — any introductory blog post is fine

SQL Basics

- Understand what a database is: a collection of organised tables
- Understand what a table is: rows (records) and columns (fields)
- Download and install DB Browser for SQLite — open it and confirm it works
- Watch: 'SQL in 7 Minutes' (YouTube) — any beginner SQL video
- Download sample e-commerce SQLite database from class resources folder

Advanced SQL

- Review your Day 4 queries — can you explain each line?
- Understand what a JOIN is conceptually: combining two tables using a common column
- Think about: what is the difference between INNER JOIN and LEFT JOIN?
- Read: 'Window Functions Explained' — Mode Analytics blog (free, Google it)

Libraries Review + Project

- Download the WHO COVID-19 dataset from Kaggle or WHO website
- Review your notes from Days 1–5 — what was the most confusing concept?
- Write down 3 questions you want answered before the week ends
- Attempt: load the COVID CSV in Pandas without any guidance — try your best

Phase A: Core Python Fundamentals

We have to build the Python foundation that every data science library depends on. Every line of NumPy, Pandas, and Scikit-learn code is ordinary Python underneath. Weak fundamentals will slow down all future learning.

Topic	Activity Type
Variables, Data Types, Operators	Lecture + Live Coding
Control Flow — if/else, loops	Guided Practice
Functions & Scope	Lecture + Exercises
Data Structures — Lists, Dicts, Sets, Tuples	Hands-on Lab
OOP Basics + File Handling + Error Handling	Mini Project

1 — Variables, Data Types & Operators

Core concept: Everything in Python is an object. Every value has a type, and type determines what operations are allowed.

Topics to Cover in Detail:

- **Variables and assignment: name = value, Python is dynamically typed (no need to declare type)**
  - int: whole numbers — age = 25, count = 0
  - float: decimals — salary = 75000.50, pi = 3.14159
  - str: text in quotes — name = 'Alice', city = "Mumbai"
  - bool: True or False — is_active = True, has_degree = False
  - NoneType: absence of value — result = None
- **Type checking and conversion: type(), int(), float(), str(), bool()**
  - type(25) → <class 'int'>, str(25) → '25', int('42') → 42
- **Arithmetic operators: + - * / // % ****
  - Integer division: 17 // 5 = 3, Modulo: 17 % 5 = 2, Power: 2**10 = 1024
- **Comparison operators: == != > < >= <= (return True or False)**
- **Logical operators: and, or, not**
- **String operations: concatenation (+), repetition (*), f-strings for formatting**
  - f'Hello {name}, you are {age} years old' — industry standard
- **Multiple assignment and swap: a, b = 5, 10 then a, b = b, a**

Control Flow: if/else and Loops

Topics to Cover in Detail:

- **if / elif / else — decision branching based on conditions**
  - Indentation is NOT optional — Python uses it to define blocks (4 spaces)
  - Nested if: checking conditions inside conditions
- **while loops — repeat as long as condition is True; always need a way to exit**
  - Infinite loop danger: always include a counter or break condition
- **for loops — iterate over any sequence (list, string, range, dict)**
  - range(start, stop, step): range(0, 10, 2) → 0, 2, 4, 6, 8
  - Looping over a list: for item in my_list:
  - Looping with index: for i, val in enumerate(my_list):
- **Loop control: break (exit loop), continue (skip to next), pass (do nothing)**
- **List comprehensions — Pythonic one-line loop for building lists**
  - squares = [x**2 for x in range(1, 11)]
  - evens = [x for x in range(20) if x % 2 == 0]
  - This is CRITICAL for data science — Pandas uses this pattern heavily

Functions & Scope

Topics to Cover in Detail:

- **Defining functions with def — the DRY principle (Don't Repeat Yourself)**
  - Parameters vs arguments: def greet(name): is parameter; greet('Alice') is argument
- **return statement — what a function gives back; without return, it returns None**
- **Default parameter values: def power(base, exponent=2): — exponent is optional**
- ***args — accepts any number of positional arguments as a tuple**
- ****kwargs — accepts any number of keyword arguments as a dictionary**
  - Used everywhere in Pandas: pd.read_csv('file.csv', sep=';', header=0)
- **Variable scope: Local (inside function) vs Global (outside function)**
  - global keyword to modify global variable from inside function
- **Lambda functions: small anonymous functions — lambda x: x**2**
  - Used with map(), filter(), sorted(), apply() in Pandas
- **Docstrings: triple-quoted strings explaining what a function does — professional habit**

Data Structures in Depth

Structure	Ordered?	Mutable?	Duplicates?	Key Data Science Use
List []	Yes	Yes	Yes	Storing a column of values, iteration
Tuple ()	Yes	No	Yes	Fixed records, function return pairs

Structure	Ordered?	Mutable?	Duplicates?	Key Data Science Use
Dictionary {}	Yes (Py 3.7+)	Yes	Keys: No	Mapping labels to values, JSON data
Set {}	No	Yes	No	Removing duplicates, set operations

- **List methods: append(), insert(), remove(), pop(), sort(), reverse(), index(), count(), extend()**
- **List slicing: my_list[start:stop:step] — negative indices work (my_list[-1] = last item)**
- **Dictionary methods: .keys(), .values(), .items(), .get(), .update(), .pop()**
  - Safe access: dict.get('key', default_value) — avoids KeyError
- **Dictionary comprehension: {k: v**2 for k, v in data.items()}**
- **Nested data structures: list of dicts is the foundation of a DataFrame in Pandas**
  - students = [{'name': 'Alice', 'score': 92}, {'name': 'Bob', 'score': 85}]
- **Set operations: union (|), intersection (&), difference (-), symmetric difference (^)**

OOP Basics + File Handling + Error Handling

OOP Basics:

- **Class definition: class Student: — blueprint for objects**
  - __init__ method: constructor, called when object is created
  - self: refers to the current object instance
- **Attributes (data) and methods (functions) inside a class**
- **Inheritance: class GradStudent(Student): — reuse parent class features**
  - Know enough to read library source code — NumPy and Pandas use OOP heavily

File Handling:

- **Read a text file: open('file.txt', 'r') — always use with statement to auto-close**
- **Write to a file: open('output.txt', 'w') — 'a' to append**
- **Read CSV manually: import csv; csv.reader()**
- **Read/write JSON: import json; json.load(), json.dump()**

Error Handling:

- **try / except block — catch errors without crashing the program**
  - try: risky code except ValueError as e: handle the error finally: always runs
- **Common exceptions: ValueError, TypeError, KeyError, IndexError, FileNotFoundError**
- **raise Exception('message') — deliberately raise your own errors**

Phase B: NumPy & Scientific Computing

NumPy is the engine beneath Pandas, Scikit-learn, TensorFlow, and every other data science library. Understanding it means understanding how data is stored, transformed, and computed at speed.

Topic	Activity Type
NumPy Arrays vs Python Lists — Why NumPy?	Lecture + Benchmarking Demo
Array Creation, Indexing, Slicing	Guided Coding
Array Operations, Broadcasting, Universal Functions	Lab Exercises
Linear Algebra with NumPy	Lecture + Math-to-Code
Random Sampling + Practical Applications	Mini Project

1 — Why NumPy? Arrays vs Python Lists

- **Python lists are general-purpose — can hold mixed types but are slow for math**
- **NumPy arrays are homogeneous (one type) — stored in contiguous memory — 50–100× faster**
- **import numpy as np — industry-standard alias, never deviate from this**
- **dtype: the data type of array elements — float64, int32, bool, complex128**
  - arr = np.array([1.0, 2.5, 3.7], dtype=np.float32) — saves memory for large data
- **ndim, shape, size, itemsize — understanding array dimensions and memory**
  - arr.shape → (3, 4) means 3 rows, 4 columns; arr.size → 12 total elements

2 — Array Creation, Indexing & Slicing

- **np.array(), np.zeros(), np.ones(), np.full(), np.eye() (identity matrix)**
- **np.arange(start, stop, step) — like range() but returns array**
- **np.linspace(start, stop, num) — evenly spaced values including endpoint**
  - np.linspace(0, 1, 100) → 100 values from 0 to 1 — used for plotting, simulations
- **Indexing: arr[row, col] for 2D; arr[0] for first row; arr[:, 2] for 3rd column**
- **Slicing: arr[1:4, 0:2] — rows 1–3, columns 0–1**
- **Boolean indexing: arr[arr > 5] — returns all elements greater than 5 — CRITICAL for filtering**
  - This is how Pandas filters data: df[df['age'] > 30] uses this exact mechanism
- **Fancy indexing: select multiple non-contiguous elements by passing an index array**
- **Copying vs viewing: arr.copy() creates independent copy; slices are views (edits affect original)**

3 — Operations, Broadcasting & Universal Functions

- **Element-wise arithmetic: arr1 + arr2, arr1 * arr2 — operates on every element simultaneously**
- **Broadcasting rules: smaller arrays are 'stretched' to match larger arrays during operations**
  - arr (4,3) + row_vector (1,3) → adds the vector to every row — no loop needed
- **Universal functions (ufuncs): np.sqrt(), np.exp(), np.log(), np.sin(), np.abs()**
- **Aggregation: np.sum(), np.mean(), np.std(), np.min(), np.max() — with axis parameter**
  - np.sum(arr, axis=0) → sum along columns; axis=1 → sum along rows
- **np.where(condition, value_if_true, value_if_false) — vectorised if/else**
  - np.where(scores >= 50, 'Pass', 'Fail') — no loop, instant for millions of rows
- **Reshaping: arr.reshape(rows, cols), arr.flatten(), arr.T (transpose)**
- **Stacking: np.hstack(), np.vstack(), np.concatenate() — combining arrays**

4 — Linear Algebra with NumPy

- **Matrix multiplication: np.dot(A, B) or A @ B — the @ operator is modern Python 3.5+**
- **np.linalg.det() — determinant; np.linalg.inv() — inverse of matrix**
- **np.linalg.eig() — eigenvalues and eigenvectors (used in PCA)**
- **np.linalg.svd() — Singular Value Decomposition (used in recommendation systems)**
- **np.linalg.solve(A, b) — solve system of linear equations Ax = b**
- **np.linalg.norm() — L1 and L2 norms; used in regularisation and distance metrics**

5 — Random Sampling + Applications

- **np.random.seed(42) — set seed for reproducibility — ALWAYS do this in data science**
- **np.random.rand(), np.random.randn() — uniform vs standard normal distribution**
- **np.random.randint(low, high, size), np.random.choice(array, size, replace)**
- **np.random.shuffle() — in-place; np.random.permutation() — returns new shuffled array**
- **Simulate coin flips, dice rolls, random sampling for statistical experiments**
- **Practical: implement gradient descent from scratch using only NumPy**

Phase C : Pandas — Data Manipulation Mastery

Pandas is the most-used data science library in Python. A data scientist spends 60–70% of their time in Pandas — loading data, cleaning it, transforming it, and summarising it. Today we master it completely.

Topic	Activity Type
Series & DataFrame fundamentals + Loading Data	Lecture + Demo
Indexing, Selection & Filtering	Guided Lab
Data Cleaning — Nulls, Duplicates, Types	Hands-on with Titanic
GroupBy, Aggregation & Pivot Tables	Group Project
Merging, Joining & Applying Functions	Capstone EDA

1 — Series, DataFrame & Loading Data

- **Series: 1-dimensional labelled array — like a single column in a spreadsheet**
  - s = pd.Series([10, 20, 30], index=['a', 'b', 'c']) — custom or default integer index
- **DataFrame: 2-dimensional labelled table — the heart of Pandas**
  - df = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30], 'Score': [88, 92]})
- **Loading data: pd.read_csv(), pd.read_excel(), pd.read_json(), pd.read_sql()**
  - Key parameters: sep, header, index_col, usecols, dtype, parse_dates, nrows, encoding
- **First look at data: df.head(10), df.tail(), df.info(), df.describe(), df.shape, df.columns**
- **Data types in Pandas: int64, float64, object (string), bool, datetime64, category**

2 — Indexing, Selection & Filtering

- **Column selection: df['column'] → Series; df[['col1', 'col2']] → DataFrame**
- **.loc[] — label-based: df.loc[0:5, 'Name':'Age'] — inclusive on both ends**
- **.iloc[] — integer-position-based: df.iloc[0:5, 0:3] — exclusive end like Python slicing**
- **Boolean filtering: df[df['Age'] > 25] — returns rows where condition is True**
  - Multiple conditions: df[(df['Age'] > 25) & (df['Score'] >= 80)] — use & not 'and'
  - df[df['City'].isin(['Mumbai', 'Delhi', 'Bangalore'])] — filter by list of values
- **df.query('Age > 25 and Score >= 80') — SQL-like syntax for filtering**
- **df.at[row, col] — fast single value access; df.iat[i, j] — by integer position**
- **Setting values: df.loc[condition, 'column'] = new_value — modifying specific cells**

3 — Data Cleaning (On Titanic Dataset)

- **Detecting missing values: df.isnull().sum(), df.isnull().mean() — find % missing per column**

- **Handling nulls:**
  - df.dropna() — remove rows/columns with any null
  - df.fillna(value) — fill with a constant, mean, median, mode, or forward/backward fill
  - df['Age'].fillna(df['Age'].median(), inplace=True) — real-world strategy
- **Removing duplicates: df.duplicated().sum(), df.drop_duplicates(subset=['col'], keep='first')**
- **Changing data types: df['Date'] = pd.to_datetime(df['Date'], format='%Y-%m-%d')**
  - df['Category'] = df['Category'].astype('category') — saves memory on repeated string values
- **Renaming columns: df.rename(columns={'old': 'new'}), df.columns = ['a', 'b', 'c']**
- **String cleaning: df['Name'].str.strip(), .str.lower(), .str.replace(), .str.split()**
- **Detecting and handling outliers: IQR method — values below Q1-1.5*IQR or above Q3+1.5*IQR**

4 — GroupBy, Aggregation & Pivot Tables

- **df.groupby('column') — splits data into groups by unique values of column**
  - df.groupby('Pclass')['Survived'].mean() — survival rate by passenger class
  - df.groupby(['Pclass', 'Sex'])['Survived'].agg(['mean', 'sum', 'count'])
- **Named aggregations: df.groupby('col').agg(avg_age=('Age', 'mean'), total=('Revenue', 'sum'))**
- **transform() — applies aggregation and returns result aligned to original index**
  - df['mean_class_age'] = df.groupby('Pclass')['Age'].transform('mean')
- **Pivot tables: pd.pivot_table(df, values='Survived', index='Pclass', columns='Sex', aggfunc='mean')**
  - Equivalent to Excel pivot tables but in code — reproducible and scalable
- **df.value_counts(), df['col'].value_counts(normalize=True) — frequency distributions**
- **df.crosstab() — frequency table between two categorical variables**

5 — Merging, Joining & Applying Functions

- **pd.merge(left, right, on='key', how='inner/left/right/outer') — joins two DataFrames**
  - on= : column name to join on; left_on= / right_on= when column names differ
- **pd.concat([df1, df2], axis=0) — stack rows; axis=1 — add columns side by side**
- **df.join() — join by index (faster when index is already the key)**
- **df.apply(func, axis=0/1) — apply any Python function to rows or columns**
  - df['Title'] = df['Name'].apply(lambda x: x.split(',')[1].split('.')[0].strip())
- **df['col'].map(dict) — replace values using a mapping dictionary**
  - df['Sex_num'] = df['Sex'].map({'male': 0, 'female': 1})
- **Sorting: df.sort_values('Age', ascending=False), df.sort_values(['col1', 'col2'])**
- **df.reset_index(), df.set_index('column') — managing the index**

Phase D: SQL — Beginner to Intermediate

SQL is not optional. In most companies, the data a data scientist needs lives in a relational database. You cannot fetch, join, filter, or aggregate that data without SQL. Today we build every skill from SELECT to advanced JOINs.

Topic	Activity Type
Relational Database Theory + Basic SELECT	Lecture + DB Browser Demo
Filtering, Sorting, Aggregation	Guided Query Writing
GROUP BY, HAVING, Subqueries	Lab Exercises
JOINs — All Types in Depth	Visual Diagramming + Practice
String Functions, Date Functions, CASE WHEN	E-Commerce DB Project

1 — Databases, Tables & Basic SELECT

- **What is a relational database? Tables with rows and columns linked by keys**
- **Primary key: unique identifier for each row — no duplicates, no nulls**
- **Foreign key: column that references the primary key of another table — creates a relationship**
- **SQL is NOT case-sensitive for keywords, but convention is to write them in UPPER CASE**
- **Every SQL statement ends with a semicolon ;**

Clause	Purpose	Example
SELECT	Choose which columns to display	SELECT customer_name, total_amount
FROM	Specify the source table	FROM orders
WHERE	Filter rows by condition	WHERE total_amount > 500
ORDER BY	Sort the results	ORDER BY order_date DESC
LIMIT	Return only N rows	LIMIT 10
DISTINCT	Remove duplicate rows	SELECT DISTINCT country FROM customers
AS	Create an alias for column or table	SELECT total * 1.18 AS total_with_tax

2 — Filtering, Sorting & Aggregation

- **WHERE operators: =, !=, >, <, >=, <=, BETWEEN, IN, NOT IN, LIKE, IS NULL, IS NOT NULL**

    - BETWEEN: WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31' — inclusive
    - IN: WHERE category IN ('Electronics', 'Clothing', 'Books')
    - LIKE: WHERE name LIKE 'A%' — starts with A; '%son' — ends with son; '%ar%' — contains ar
    - IS NULL: WHERE email IS NULL — finds missing values
- **Logical operators: AND, OR, NOT — AND has higher precedence than OR; use parentheses**
- **Aggregate functions: COUNT(), SUM(), AVG(), MIN(), MAX()**
    - COUNT(*) counts all rows; COUNT(column) counts non-null values in that column
    - COUNT(DISTINCT product_id) — count unique products ordered
- **ORDER BY: ASC (default, small to large), DESC (large to small)**
    - ORDER BY total DESC, customer_name ASC — multi-column sort

3 — GROUP BY, HAVING & Subqueries

- **GROUP BY groups rows with the same value in a column and applies aggregate function to each group**
    - SELECT category, COUNT(*) AS num_products, AVG(price) AS avg_price FROM products GROUP BY category
- **Rule: every column in SELECT must either be in GROUP BY or be inside an aggregate function**
- **HAVING filters groups AFTER aggregation — WHERE filters rows BEFORE aggregation**
    - SELECT category, SUM(revenue) AS total FROM sales GROUP BY category HAVING SUM(revenue) > 10000
- **Subqueries — a query nested inside another query, used in WHERE, FROM, or SELECT**
    - WHERE customer_id IN (SELECT customer_id FROM orders WHERE total > 1000)
    - FROM (SELECT ...) AS subquery — derived table, requires alias
    - Correlated subquery: references the outer query — slower but sometimes necessary
- **SQL execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT**
    - This is WHY you cannot use an alias defined in SELECT inside WHERE — SELECT runs after WHERE

4 — JOINs in Depth

JOINs are the most important SQL concept for data scientists. Most real data lives across multiple tables.

JOIN Type	What It Returns	When To Use
INNER JOIN	Only rows that match in BOTH tables	Default — most common join
LEFT JOIN	All rows from left table + matching rows from right (NULLs for no match)	Keep all customers, even those with no orders
RIGHT JOIN	All rows from right table + matching rows from left (NULLs for no match)	Rare — usually rewrite as LEFT JOIN

JOIN Type	What It Returns	When To Use
FULL OUTER JOIN	All rows from both tables (NULLs where no match)	Finding all unmatched records on both sides
CROSS JOIN	Every row in table A paired with every row in table B	Generating all combinations — use carefully
SELF JOIN	A table joined to itself using aliases	Org charts, finding pairs within same table

- **Syntax: SELECT * FROM orders o INNER JOIN customers c ON o.customer_id = c.customer_id**
- **Always use aliases for table names in JOINs — improves readability**
- **Anti-join pattern: LEFT JOIN ... WHERE right_table.id IS NULL — find records with no match**
  - Customers who have NEVER placed an order: LEFT JOIN orders ON ... WHERE orders.id IS NULL
- **Joining more than two tables: chain multiple JOINs in sequence**

5 — String Functions, Date Functions & CASE WHEN

- **String functions: UPPER(), LOWER(), TRIM(), LENGTH(), SUBSTR(), REPLACE(), CONCAT()**
  - CONCAT(first_name, ' ', last_name) AS full_name — combining columns
- **Date functions vary by database — SQLite: date(), strftime(); PostgreSQL: EXTRACT(), DATE_TRUNC()**
  - strftime('%Y', order_date) → extract year; '%m' → month; '%d' → day
  - DATE('now') → today's date; DATE('now', '-30 days') → 30 days ago
- **CASE WHEN — if/else logic inside SQL**
  - CASE WHEN score >= 90 THEN 'A' WHEN score >= 80 THEN 'B' ELSE 'C' END AS grade
  - Conditional aggregation: SUM(CASE WHEN status = 'Completed' THEN amount ELSE 0 END) AS completed_revenue
- **COALESCE(col1, col2, default) — return first non-null value — essential for null handling**
- **NULLIF(a, b) — returns NULL if a = b, else returns a — avoids division by zero**
  - revenue / NULLIF(units, 0) AS avg_unit_price — safe division

Phase E : Advanced SQL + Python-SQL Integration

Topic	Activity Type
Window Functions — ROW_NUMBER, RANK, LAG, LEAD	Lecture + Live Queries
CTEs — WITH Clause + Recursive Queries	Guided Practice
Advanced Analytical Queries for Data Science	Query Writing Challenge
Python + SQL Integration (sqlite3, SQLAlchemy, pandas)	Coding Lab
Full Analytical SQL Report Project	Group Work

1 — Window Functions

Window functions perform calculations across rows related to the current row — without collapsing rows like GROUP BY. They are the most powerful SQL feature for analytics.

Window Function	What It Does	Example Use
ROW_NUMBER()	Assigns sequential row number per partition	De-duplicate: keep only first record per customer
RANK()	Ranks with gaps for ties (1,2,2,4)	Top N products with tied scores handled
DENSE_RANK()	Ranks without gaps for ties (1,2,2,3)	Percentile bands without skipping ranks
NTILE(n)	Divides into n equal buckets	Split customers into quartiles
LAG(col, n)	Value from n rows before current row	Month-over-month change calculation
LEAD(col, n)	Value from n rows after current row	Next month's target vs current
SUM() OVER()	Running total up to current row	Cumulative revenue by date
AVG() OVER()	Moving average across a window	7-day rolling average of sales
FIRST_VALUE()	First value in the partition window	Baseline comparison (first month's sales)

- **Syntax: FUNCTION() OVER (PARTITION BY col ORDER BY col ROWS BETWEEN ...)**
- **PARTITION BY: defines the group — resets the window for each group (like GROUP BY but keeps rows)**

- **ORDER BY inside OVER: defines the order for running totals, rank, and LAG/LEAD**
- **Frame clause: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW — cumulative sum**
  - ROWS BETWEEN 6 PRECEDING AND CURRENT ROW — 7-day rolling average

2 — CTEs (Common Table Expressions)

- **WITH clause defines a named temporary result set used in the main query**
  - Makes complex queries readable by breaking them into logical steps
  - WITH high_value_customers AS (SELECT customer_id, SUM(total) AS spend FROM orders GROUP BY 1 HAVING spend > 5000) SELECT * FROM customers JOIN high_value_customers USING (customer_id)
- **Multiple CTEs: WITH cte1 AS (...), cte2 AS (...) SELECT ... — chain as many as needed**
- **CTEs vs subqueries: CTEs are more readable and can be referenced multiple times in same query**
- **Recursive CTEs: CTE that references itself — used for hierarchical/tree data (org charts, paths)**
- **CTE vs temp table: CTE lives only for that query; temp table persists for the session**

3 — Advanced Analytical SQL for Data Science

- **Cohort analysis: group users by their sign-up month, track retention over time**
- **Funnel analysis: count how many users pass through each step in a sequence**
- **RFM analysis: Recency (last order), Frequency (order count), Monetary (total spend)**
- **Sessionisation: group user events into sessions using LAG to find time gaps > 30 minutes**
- **Data quality checks: write SQL to find nulls, duplicates, impossible values, broken referential integrity**
- **Pivoting data: transform rows into columns using conditional aggregation (CASE WHEN + GROUP BY)**

4 — Python + SQL Integration

- **sqlite3 (built-in): connect to SQLite database directly from Python**
  - conn = sqlite3.connect('ecommerce.db'); cursor = conn.cursor()
  - cursor.execute('SELECT * FROM orders WHERE total > ?', (500,)) — parameterised queries (safe)
- **Read SQL query into Pandas DataFrame: pd.read_sql_query(query, conn)**
  - df = pd.read_sql_query('SELECT * FROM orders', conn) — instant DataFrame from any SQL query
- **Write DataFrame to SQL table: df.to_sql('table_name', conn, if_exists='replace', index=False)**
- **SQLAlchemy: professional ORM for connecting to PostgreSQL, MySQL, and other production databases**
  - engine = create_engine('postgresql://user:password@host:5432/dbname')
  - pd.read_sql(query, engine) — same interface, works with all major databases
- **Best practice: always close connections; use context managers (with statement)**
- **Parameterised queries prevent SQL injection — NEVER use string formatting for user input**

Phase – F Python Libraries Review + Capstone Project

Today consolidates the entire week. Students review Matplotlib and Seaborn for visualisation, then apply everything learned to a real COVID-19 dataset. This is the most important day — applying skills to messy, real-world data.

Topic	Activity Type
Matplotlib + Seaborn — Full Visualisation Review	Lecture + Gallery Demo
SciPy & Statsmodels — Statistics in Python	Guided Exercises
Capstone: COVID-19 Data Analysis Project	Independent Project

1 — Matplotlib & Seaborn Deep Dive

Matplotlib — Full Control Plotting:

- **Figure and Axes architecture: fig, ax = plt.subplots(nrows, ncols, figsize=(12, 6))**
- **Line plot: ax.plot(x, y, color='teal', linewidth=2, marker='o', linestyle='--')**
- **Scatter: ax.scatter(x, y, c=colors, s=sizes, alpha=0.6, cmap='viridis')**
- **Bar chart: ax.bar(x, height, color='steelblue', edgecolor='black')**
- **Histogram: ax.hist(data, bins=30, density=True, alpha=0.7)**
- **Labels and formatting: ax.set_xlabel(), set_ylabel(), set_title(), legend(), grid(True)**
- **Saving: plt.savefig('chart.png', dpi=300, bbox_inches='tight') — for reports**
- **Multiple subplots: each ax in the array is an independent plot with its own axes**

Seaborn — Statistical Visualisation:

- **sns.histplot(df['Age'], kde=True) — histogram with kernel density estimate overlay**
- **sns.boxplot(x='Pclass', y='Age', data=df) — distribution by category**
- **sns.violinplot() — like boxplot but shows full distribution shape**
- **sns.heatmap(df.corr(), annot=True, cmap='coolwarm', vmin=-1, vmax=1) — correlation matrix**
- **sns.pairplot(df, hue='Survived') — scatter matrix for all numeric columns**
- **sns.countplot(x='Pclass', hue='Survived', data=df) — count with colour split**

2 — SciPy & Statsmodels

- **scipy.stats: t-tests, chi-square, ANOVA, Mann-Whitney, Kolmogorov-Smirnov tests**
  - t, p = scipy.stats.ttest_ind(group1, group2) — does mean differ between two groups?
  - chi2, p, dof, expected = scipy.stats.chi2_contingency(contingency_table)
- **Interpreting p-value: p < 0.05 → statistically significant at 95% confidence level**
- **scipy.stats.norm.pdf(), .cdf() — compute probabilities from normal distribution**

- **Statsmodels OLS: fit a linear regression and get the full statistical summary**
  - import statsmodels.api as sm; model = sm.OLS(y, sm.add_constant(X)).fit(); print(model.summary())
- **The summary gives: R-squared, F-statistic, coefficient estimates, p-values, confidence intervals**

s 3–5 —PROJECT: COVID-19 Data Analysis

Project Brief

Using the WHO COVID-19 dataset (or Johns Hopkins dataset from Kaggle), students will independently perform a complete end-to-end data analysis covering all skills from the week. The instructor acts as a guide only — students lead their own analysis.

Step 1 — Data Loading & Inspection (30 minutes)

- Load all CSV files into Pandas DataFrames; check shapes, column names, data types
- Identify missing values — what % of each column is missing? What is the pattern?
- Check date columns — convert to datetime type; extract year, month, week

Step 2 — Data Cleaning (30 minutes)

- Handle missing values with appropriate strategy (fill, drop, or flag)
- Remove duplicate records; verify country names are consistent (UK vs United Kingdom)
- Calculate derived columns: cases per million, deaths per million, case fatality rate (%)

Step 3 — Exploratory Analysis (45 minutes)

- Top 10 countries by total confirmed cases — bar chart
- Daily new cases globally over time — line chart; add a 7-day rolling average
- Correlation between cases and deaths — scatter plot with regression line
- Distribution of case fatality rates across countries — histogram with KDE

Step 4 — Country Comparison (30 minutes)

- Compare India, USA, UK, Brazil, Germany on: total cases, peak daily cases, fatality rate
- Use a pivot table: countries as rows, months as columns, total new cases as values
- Use a heatmap to visualise the pivot table

Step 5 — Hypothesis Testing (15 minutes)

- Formulate a hypothesis: e.g., 'Continents with higher GDP have lower fatality rates'
- Use scipy.stats to run the appropriate test; interpret the p-value

Step 6 — Summary Report (30 minutes)

- Write 5 key findings in plain English — what does the data tell us?
- Save all plots to files; create a markdown or notebook report
- Present findings to the class: 2 minutes per student, 2 charts, 3 bullet points

COMPLETE SUMMARY

Core Topic	Python Concepts	SQL Concepts	Project / Output
Core Python	Variables, loops, functions, OOP, file handling, error handling	—	BankAccount class + data filter program
NumPy	Arrays, indexing, broadcasting, ufuncs, linear algebra, random	—	Matrix operations + gradient descent simulation
Pandas	Series, DataFrame, groupby, merge, apply, pivot, EDA workflow	—	Full Titanic EDA notebook
SQL Basics–Intermediate	—	SELECT, WHERE, GROUP BY, HAVING, JOIN, CASE, string/date functions	20 e-commerce SQL queries
Advanced SQL + Integration	sqlite3, SQLAlchemy, pd.read_sql	Window functions, CTEs, cohort analysis, RFM	Analytical SQL report + Python dashboard
Libraries + Capstone	Matplotlib, Seaborn, SciPy, Statsmodels	—	Complete COVID-19 analysis report

LEARNING OUTCOMES — By This Week, you Will Be Able To:

- Write clean, functional Python code using all core language features
- Manipulate arrays and matrices with NumPy at speed — no loops for numerical operations
- Load, clean, filter, group, merge, and analyse any tabular dataset using Pandas
- Write SQL queries from basic SELECT to advanced window functions and CTEs
- Connect Python to SQL databases and run queries that feed directly into Pandas
- Create publication-quality visualisations using Matplotlib and Seaborn
- Apply statistical tests to validate data-driven hypotheses
- Complete an end-to-end exploratory data analysis on a real-world dataset